CONVOLUTIONAL ENCODER

sarathkcofficial@outlook.com

Design and Implementation of a Convolutional Encoder Using C Programming

Aim

To implement a convolutional encoder in the C programming language using shift registers and XOR operations, and to generate encoded output bits from a given binary input sequence.

Objectives

- To understand the basic principle of convolutional encoding.
- To implement a convolutional encoder using C.
- To understand the role of generator polynomials and shift registers.
- To generate two output bits for every input bit.
- To execute and verify the program using a Linux terminal or Windows Subsystem for Linux (WSL).

Introduction

Convolutional encoding is a forward error correction technique used in digital communication systems to improve the reliability of transmitted data.

In a communication system, transmitted bits may be corrupted by noise or interference. A convolutional encoder adds redundancy to the input data so that a receiver can detect and correct certain transmission errors.

Unlike a block encoder, which processes a fixed block of input bits independently, a convolutional encoder generates output bits based on the current input bit and previously stored input bits.

The encoder uses shift registers and modulo-2 addition, which is implemented using the XOR operation in digital logic.

Theory

Parameters of the Encoder

The convolutional encoder implemented in this experiment uses the following parameters:

Parameter	Value
Constraint length ((K))	3

Memory elements	2
Generator polynomial (G_1)	111
Generator polynomial (G_2)	101
Code rate ((R))	1/2
Initial register state	00

Code Rate

The code rate represents the ratio of the number of input bits to the number of output bits.

For this encoder, each input bit generates two output bits.

Therefore, four input bits generate eight encoded output bits.

Shift Register Operation

The encoder contains two memory elements, represented by reg1 and reg2.

- current represents the present input bit.
- reg1 stores the previous input bit.
- reg2 stores the input bit from two time steps earlier.

The registers are initially set to zero.

After generating the output bits, the registers are updated to store the current and previous input bits.

Generator Polynomials

The encoder uses two generator polynomials.

Generator 1: (G_1=111)

The first output is calculated using the current input bit and both register values.

Generator 2: (G_2=101)

The second output is calculated using the current input bit and the second register value.

The XOR truth table is:

A	**B**	**A XOR B**
0	0	0
0	1	1
1	0	1
1	1	0

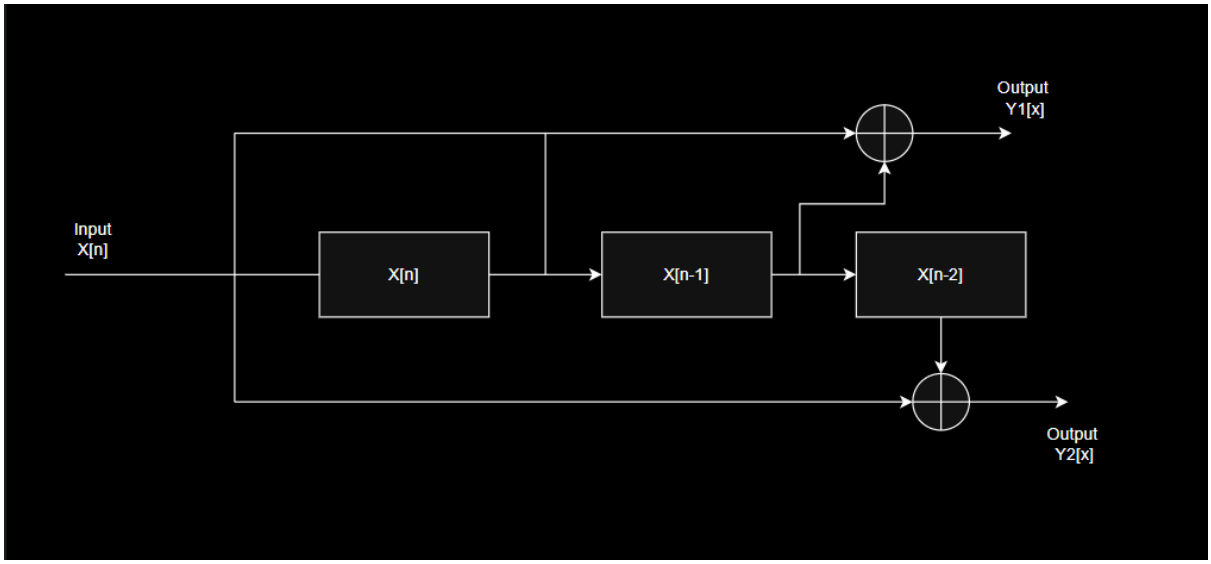


Fig. 1. Convolutional Encoder Block diagram

Algorithm

1. Start the program.
2. Read the number of input bits.
3. Read the binary input sequence.
4. Initialize both shift-register values to zero.
5. Read the current input bit.
6. Calculate the first output bit using generator polynomial 111.
7. Calculate the second output bit using generator polynomial 101.
8. Display the two output bits.
9. Update the shift registers.
10. Repeat the process until all input bits are processed.
11. Stop the program.

C Code:

```
#include <stdio.h>

int main()
{
   int input[100];
   char bits[100];
        int n;
   int reg1 = 0, reg2 = 0;
   int i;
        int current, out1, out2;
```

```
    // Read the number of input bits
    printf("Enter the number of input bits: ");
    scanf("%d", &n);

    // Read the binary input as a string
    printf("Enter the input bits (0 or 1): ");
    scanf("%99s", bits);

    // Convert characters into integer bits
    for (i = 0; i < n; i++)
    {
        input[i] = bits[i] - '0';
    }

    printf("\nEncoded output: ");

    // Process each input bit
    for (i = 0; i < n; i++)
    {
        current = input[i];

        // First output: Generator 111
        out1 = current ^ reg1 ^ reg2;

        // Second output: Generator 101
        out2 = current ^ reg2;

        // Display encoded bits
        printf("%d%d ", out1, out2);

        // Update the shift registers
        reg2 = reg1;
        reg1 = current;
    }

    printf("\n");

    return 0;
}
```

Example execution of the code

```
skc@LAPTOP-4EF8D6DH:~/c_programming/convolution_encoder$ ./convo
Enter the number of input bits: 5
Enter the input bits (0 or 1): 1 0 1 1 0

Encoded output: 11 10 00 01 01
```

Fig. 2. C Program result

Result

The convolutional encoder was successfully implemented using C programming. The program accepts a binary input sequence and generates two encoded output bits per input bit using the specified generator polynomials and shift-register operations.

The implementation demonstrates the fundamental operation of a rate-1/2 convolutional encoder.

➢ To design and verify a rate-1/2 convolutional encoder using Verilog RTL, with constraint length (K=3) and generator polynomials (G1=111) and (G2=101)

Objectives

- Understand convolutional encoding and its role in error-control coding.
- Implement shift registers and XOR operations using Verilog.
- Generate two encoded output bits for every valid input bit.
- Verify the RTL design using a simulation testbench.

Introduction

Convolutional encoding is a forward error correction technique used in digital communication systems. It introduces redundancy into transmitted data so that a receiver can recover information more reliably when transmission errors occur.

A convolutional encoder generates output bits based on the current input bit and previously stored input bits. The encoder can be implemented in digital hardware using shift registers and XOR gates.

In this experiment, a rate-1/2 convolutional encoder is implemented using Verilog RTL. The design contains two memory elements and uses two generator polynomials to produce the encoded outputs.

Design Specifications

Parameter	**Specification**
Constraint length	3
Memory elements	2
Generator polynomial	111
Generator polynomial	101
Code rate	1/2
Clock	Rising-edge triggered
Reset	Active-high synchronous
Input interface	One bit per valid clock cycle

Output interface	Two encoded bits
Initial state	00

Theory

Same as mention in the C program code.

RTL Implementation

```
module convolution_encoder (
    input  wire       clk,
    input  wire       rst,
    input  wire       valid_in,
    input  wire       data_in,
    output reg  [1:0] data_out,
    output reg        valid_out
);

    // Two memory elements
    reg reg1;
    reg reg2;

    always @(posedge clk) begin
        if (rst) begin
            reg1      <= 1'b0;
            reg2      <= 1'b0;
            data_out  <= 2'b00;
            valid_out <= 1'b0;
        end
        else begin
            valid_out <= 1'b0;

            if (valid_in) begin
                // Generator G1 = 111
                data_out[1] <= data_in ^ reg1 ^ reg2;

                // Generator G2 = 101
```

```
                data_out[0] <= data_in ^ reg2;

                // Update shift registers
                reg2 <= reg1;
                reg1 <= data_in;

                valid_out <= 1'b1;
            end
        end
    end

endmodule
```

Testbench :

```
`timescale 1ns/1ps

module tb_convolution_encoder;

    reg clk;
    reg rst;
    reg valid_in;
    reg data_in;

    wire [1:0] data_out;
    wire valid_out;

    reg [4:0] test_data;
    reg [9:0] expected_output;

    integer i;
    integer errors;

    convolution_encoder dut (
        .clk(clk),
        .rst(rst),
        .valid_in(valid_in),
```

```
    .data_in(data_in),
    .data_out(data_out),
    .valid_out(valid_out)
);

// 10 ns clock period
always #5 clk = ~clk;

initial begin
    $dumpfile("convolution.vcd");
    $dumpvars(0, tb_convolution_encoder);

    clk = 0;
    rst = 1;
    valid_in = 0;
    data_in = 0;
    test_data = 5'b10110;

    // Expected pairs: 11 10 00 01 01
    expected_output = 10'b1110000101;
    errors = 0;

    // Apply reset
    repeat (2) @(negedge clk);
    rst = 0;

    $display("Input | Actual Output | Expected Output | Status");
    $display("-----------------------------------------------");

    // Send bits MSB first
    for (i = 4; i >= 0; i = i - 1) begin
        @(negedge clk);
        data_in = test_data[i];
        valid_in = 1;
```

```
            @(posedge clk);
            #1;

            $write("   %b  |      %b      |       %b       | ",
                   data_in, data_out,
                   expected_output[i*2 +: 2]);

            if (data_out === expected_output[i*2 +: 2]) begin
                $display("PASS");
            end
            else begin
                $display("FAIL");
                errors = errors + 1;
            end
        end

        @(negedge clk);
        valid_in = 0;
        if (errors == 0)
            $display("PASS: All 5 input bits encoded correctly.");
        else
            $display("FAIL: %0d output mismatches.", errors);

        $display("Expected encoded sequence: 11 10 00 01 01");

        $finish;
    end

endmodule
```

Results

Compilation

```
skc@LAPTOP-4EF8D6DH:~/sandbox_verilog/convolutional_Encoder$ iverilog -o sim con_encoder.v tb_convo_encoder.v
skc@LAPTOP-4EF8D6DH:~/sandbox_verilog/convolutional_Encoder$ vvp sim
VCD info: dumpfile convolution.vcd opened for output.
Input | Actual Output | Expected Output | Status
-------------------------------------------------
   1  |      11       |       11        | PASS
   0  |      10       |       10        | PASS
   1  |      00       |       00        | PASS
   1  |      01       |       01        | PASS
   0  |      01       |       01        | PASS
-------------------------------------------------
PASS: All 5 input bits encoded correctly.
Expected encoded sequence: 11 10 00 01 01
tb_convo_encoder.v:87: $finish called at 80000 (1ps)
```

Fig. 3. RTL compilation results

Simulation

